

Essential attributes of good software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software engineering ethics

- ✧ Software engineering involves wider responsibilities than simply the application of technical skills.
- ✧ Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- ✧ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

Issues of professional responsibility

- ✧ **Confidentiality**
 - Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- ✧ **Competence**
 - Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.
- ✧ **Intellectual property rights**
 - Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.
- ✧ **Computer misuse**
 - Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

Application types

- ✧ **Stand-alone applications**
 - These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- ✧ **Interactive transaction-based applications**
 - Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- ✧ **Embedded control systems**
 - These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

- ✧ **Batch processing systems**
 - These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- ✧ **Entertainment systems**
 - These are systems that are primarily for personal use and which are intended to entertain the user.
- ✧ **Systems for modeling and simulation**
 - These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

Software process models

- ✧ The waterfall model
 - Plan-driven model. Separate and distinct phases of specification and development.
- ✧ Incremental development
 - Specification, development and validation are interleaved. May be plan-driven or agile.
- ✧ Integration and configuration
 - The system is assembled from existing configurable components. May be plan-driven or agile.
- ✧ In practice, larger systems are developed using a process that incorporates elements from all of these models.
- ✧

Incremental development benefits

- ✧ The cost of accommodating changing customer requirements is reduced.
 - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
- ✧ It is easier to get customer feedback on the development work that has been done.
 - Customers can comment on demonstrations of the software and see how much has been implemented.
- ✧ More rapid delivery and deployment of useful software to the customer is possible.
 - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

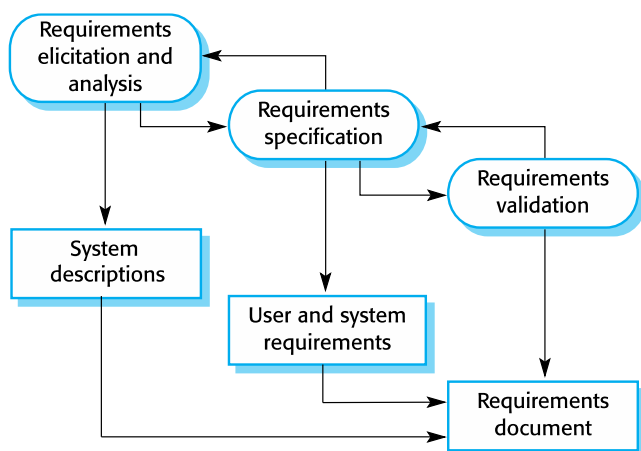
Incremental development problems

- ✧ The process is not visible.
 - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
- ✧ System structure tends to degrade as new increments are added.
 - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

Types of requirement

- ✧ User requirements
 - Statements in natural language plus diagrams of the services the system provides and its operational constraints. Written for customers.
- ✧ System requirements
 - A structured document setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be part of a contract between client and contractor.

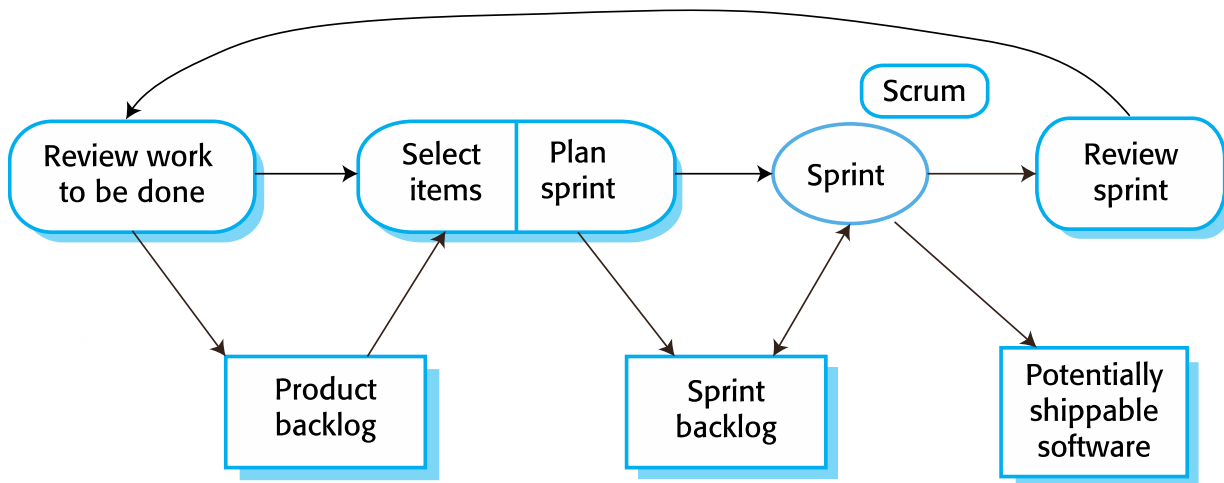
The requirements engineering process



Contrast scrum vs plan based development (3 preceptions)

1. ****Flexibility vs. Rigidity****: Scrum emphasizes adaptability and embraces change throughout the development process, while plan-based development follows a fixed plan and resists alterations once established.
2. ****Iterative vs. Sequential****: Scrum breaks down projects into smaller, manageable iterations, allowing for continuous improvement and feedback, whereas plan-based development typically follows a linear, step-by-step progression from planning to execution.
3. ****Collaborative vs. Hierarchical****: Scrum fosters collaboration and communication among team members, promoting self-organization and shared decision-making, whereas plan-based development often relies on top-down management and strict adherence to predefined roles and processes.

Scrum sprint cycle



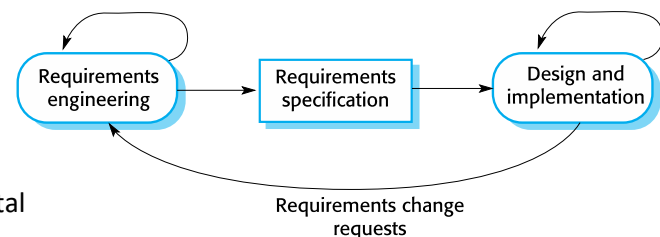
Agile development

- ✧ Program specification, design and implementation are inter-leaved
- ✧ The system is developed as a series of versions or increments with stakeholders involved in version specification and evaluation
- ✧ Frequent delivery of new versions for evaluation
- ✧ Extensive tool support (e.g. automated testing tools) used to support development.
- ✧ Minimal documentation – focus on working code

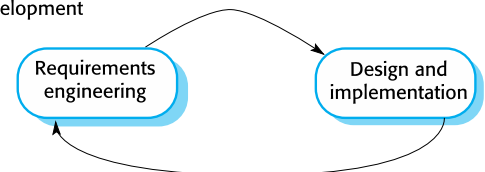
Plan-driven and agile development

- ✧ **Plan-driven development**
 - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
 - Not necessarily waterfall model – plan-driven, incremental development is possible
 - Iteration occurs within activities.
- ✧ **Agile development**
 - Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Plan-based development



Agile development



Functional and non-functional requirements

- ✧ Functional requirements
 - Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
 - May state what the system should not do.
- ✧ Non-functional requirements
 - Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
 - Often apply to the system as a whole rather than individual features or services.
- ✧ Domain requirements
 - Constraints on the system from the domain of operation

Identifying ambiguities (Scenario: Drone & Ticket Issue)

Title: Identifying Ambiguities in Requirements: A Case Study of Drone Altitude Control Issue

Abstract:

This paper presents a case study focusing on the identification of ambiguities in requirements within the context of software engineering, using a scenario involving a drone altitude control issue. Ambiguities in requirements can lead to misunderstandings, misinterpretations, and ultimately, project failures. Through this case study, we aim to demonstrate the importance of thorough requirement analysis and the techniques for identifying and resolving ambiguities effectively.

Introduction:

In software engineering, clear and unambiguous requirements are crucial for the successful development of software systems. Ambiguities in requirements can lead to costly rework, delays, and even safety risks. In this paper, we explore the process of identifying ambiguities in requirements using a real-world scenario involving a drone altitude control issue. By examining this scenario, we highlight common types of ambiguities and propose techniques for their detection and resolution.

Scenario Description:

A company is in the process of developing a new drone for aerial surveillance. However, during testing, an issue arises with the drone's altitude control system, causing it to occasionally malfunction. This poses a potential safety risk to nearby objects and people. Customers who have pre-ordered the drone are concerned and demand a resolution before the product launch.

Identification of Ambiguities:

1. Scope Ambiguity: It is unclear whether the altitude control issue affects all drones or only a specific batch. Additionally, the term "occasionally malfunction" lacks specificity, leaving room for interpretation regarding the frequency and severity of the issue.

2. Safety Impact Ambiguity: The potential danger posed by the altitude control issue is not clearly defined. Without a precise understanding of the safety implications, it is challenging to prioritize and address the issue effectively.

3. Customer Expectations Ambiguity: The demand for a resolution from customers is ambiguous. It is unclear what specific actions they expect from the development team and whether their expectations align with the technical feasibility and timeline of the project.

Conclusion:

Ambiguities in requirements can significantly impact the success of software engineering projects. Through the case study of the drone altitude control issue, we have identified common types of ambiguities and highlighted their implications. By employing systematic techniques for requirement analysis and clarification, software engineers can mitigate risks associated with ambiguities and ensure the delivery of high-quality software systems.